

SONATEL ACADEMY — ORANGE DIGITAL CENTER

Programme p1IA — Juillet 2026

ATELIER LINUX

Rapport de réalisation

Préparation d'un environnement Linux pour un projet d'IA

Maimouna Diallo

Contexte de l'atelier

Nous venons d'intégrer une entreprise ; notre première mission consiste à préparer un environnement Linux complet pour un projet d'intelligence artificielle : organiser les données, manipuler des fichiers, gérer les utilisateurs, sécuriser les fichiers, installer les outils nécessaires, récupérer un dataset, lancer des traitements, sauvegarder le projet et automatiser certaines tâches.

Partie 1 — Organisation du projet

Création de l'arborescence complète du projet IA_Project, avec les dossiers datasets/brut, datasets/clean, config, logs, scripts et models, ainsi que les fichiers de données et de configuration fournis pour l'atelier.

1.1) Créer l'arborescence du projet et les fichiers de données/configuration

Commande utilisée : `mkdir -p` pour créer les arborescences

```
wildflower@wildflower:~$ mkdir -p IA_Project/datasets/brut
wildflower@wildflower:~$ mkdir -p IA_Project/config
wildflower@wildflower:~$ mkdir -p IA_Project/logs
wildflower@wildflower:~$ mkdir -p IA_Project/scripts
wildflower@wildflower:~$ mkdir -p IA_Project/models
wildflower@wildflower:~$ cd IA_Project
wildflower@wildflower:~/IA_Project$ LS
LS: command not found
wildflower@wildflower:~/IA_Project$ ls
config  datasets  logs  models  scripts
```

commande utilisée : `echo`

```
wildflower@wildflower: ~/IA_ x + v
wildflower@wildflower:~$ cd IA_Project
wildflower@wildflower:~/IA_Project$ rm datasets/brut/readme.txt
wildflower@wildflower:~/IA_Project$ echo "Projet : IA de prédiction des achats" > datasets/brut/readme.txt
echo "" >> datasets/brut/readme.txt
echo "Ce dossier contient les données brutes destinées" >> datasets/brut/readme.txt
echo "à l'entraînement d'un modèle de Machine Learning." >> datasets/brut/readme.txt
echo "" >> datasets/brut/readme.txt
echo "Fichiers disponibles :" >> datasets/brut/readme.txt
echo "" >> datasets/brut/readme.txt
echo "- train.csv" >> datasets/brut/readme.txt
echo "- test.csv" >> datasets/brut/readme.txt
echo "- clients.csv" >> datasets/brut/readme.txt
echo "" >> datasets/brut/readme.txt
echo "Ne jamais modifier directement les fichiers originaux." >> datasets/brut/readme.txt
echo "" >> datasets/brut/readme.txt
echo "Les versions nettoyées devront être placées" >> datasets/brut/readme.txt
echo "dans datasets/clean." >> datasets/brut/readme.txt
wildflower@wildflower:~/IA_Project$ cat datasets/brut/readme.txt
Projet : IA de prédiction des achats

Ce dossier contient les données brutes destinées
à l'entraînement d'un modèle de Machine Learning.

Fichiers disponibles :

- train.csv
- test.csv
- clients.csv

Ne jamais modifier directement les fichiers originaux.

Les versions nettoyées devront être placées
dans datasets/clean.
wildflower@wildflower:~/IA_Project$ cat -n datasets/brut/readme.txt
 1  Projet : IA de prédiction des achats
 2
 3  Ce dossier contient les données brutes destinées
```

commande utilisée : printf

```
wildflower@wildflower: ~/IA
wildflower@wildflower:~/IA_Project$ printf "id,age,revenu,ville,achat\n1,25,350000,Dakar,Oui\n2,42,720000,Thies,Non\n3,31,500000,Saint-Louis,Oui\n4,28,410000,Dakar,Oui\n5,54,900000,Kaolack,Non\n6,36,620000,Ziguinchor,Oui\n7,29,470000,Thies,Oui\n8,47,850000,Dakar,Non\n9,39,650000,Louga,Oui\n10,26,390000,Matam,Non\n" > datasets/brut/train.csv
wildflower@wildflower:~/IA_Project$ printf "id,age,revenu,ville\n11,30,520000,Dakar\n12,41,710000,Thies\n13,27,430000,Louga\n14,50,910000,Saint-Louis\n15,35,610000,Ziguinchor\n" > datasets/brut/test.csv
wildflower@wildflower:~/IA_Project$ printf "client,profession,anciennete\nAli,Ingenieur,5\nFatou,Comptable,3\nMoussa,Professeur,8\nAwad,Docteur,12\nCheikh,Informaticien,2\n" > datasets/brut/clients.csv
wildflower@wildflower:~/IA_Project$ printf "PROJECT_NAME=IA_Project\nDATA_PATH=datasets/brut\nMODEL_PATH=models\nLOG_LEVEL=INFO\nAPI_PORT=8000\nAUTHOR=Equipe IA\n" > config/settings.conf
wildflower@wildflower:~/IA_Project$ printf "MODEL=RandomForest\nN_ESTIMATORS=100\nMAX_DEPTH=10\nTEST_SIZE=0.2\nRANDOM_STATE=42\n" > config/model.conf
wildflower@wildflower:~/IA_Project$ printf "2026-07-18 08:00:10 INFO Application démarrée\n2026-07-18 08:00:15 INFO Chargement du dataset\n2026-07-18 08:00:17 WARNING Valeurs manquantes détectées\n2026-07-18 08:00:20 INFO Nettoyage terminé\n2026-07-18 08:00:30 INFO Modèle prêt\n" > logs/application.log
wildflower@wildflower:~/IA_Project$ printf "Epoch 1 : Loss = 0.82\nEpoch 2 : Loss = 0.60\nEpoch 3 : Loss = 0.45\nEpoch 4 : Loss = 0.32\nEpoch 5 : Loss = 0.28\nTraining terminé.\n" > logs/training.log
wildflower@wildflower:~/IA_Project$ printf 'print("Lecture du fichier train.csv")\nprint("Suppression des valeurs manquantes")\nprint("Encodage des variables")\nprint("Normalisation")\nprint("Export terminé")\n' > scripts/preprocess.py
wildflower@wildflower:~/IA_Project$ printf 'print("Chargement des données...")\nprint("Prétraitement...")\nprint("Entraînement du modèle...")\nprint("Sauvegarde du modèle...")\n' > scripts/train.py
wildflower@wildflower:~/IA_Project$ cat -n datasets/brut/train.csv
 1 id,age,revenu,ville,achat
 2 1,25,350000,Dakar,Oui
 3 2,42,720000,Thies,Non
 4 3,31,500000,Saint-Louis,Oui
 5 4,28,410000,Dakar,Oui
 6 5,54,900000,Kaolack,Non
 7 6,36,620000,Ziguinchor,Oui
 8 7,29,470000,Thies,Oui
 9 8,47,850000,Dakar,Non
10 9,39,650000,Louga,Oui
11 10,26,390000,Matam,Non
wildflower@wildflower:~/IA_Project$ cat -n datasets/brut/clients.csv
 1 client,profession,anciennete
 2 Ali,Ingenieur,5
 3 Fatou,Comptable,3
 4 Moussa,Professeur,8
```

commande utilisée : cat

```
wildflower@wildflower:~/IA_Project$ cat > models/model_info.txt << 'EOF'
Nom du modèle : Random Forest
Version : 1.0
Date : 18/07/2026
Accuracy : 94 %
Auteur : Equipe IA
EOF
wildflower@wildflower:~/IA_Project$ cat -n models/model_info.txt
 1 Nom du modèle : Random Forest
 2 Version : 1.0
 3 Date : 18/07/2026
 4 Accuracy : 94 %
 5 Auteur : Equipe IA
```

commande on peut utiliser nano pour créer un fichier qui s'ouvre sur un editeur directement

Partie 2 — Manipulation des fichiers

1) Affichez le contenu de readme.txt

Commande(s) utilisée(s) : `cat`

```
wildflower@wildflower:~/IA_Project$ cat datasets/brut/readme.txt

Projet : IA de prédiction des achats

Ce dossier contient les données brutes destinées
à l'entraînement d'un modèle de Machine Learning.

Fichiers disponibles :

- train.csv
- test.csv
- clients.csv

Ne jamais modifier directement les fichiers originaux.

Les versions nettoyées devront être placées
dans datasets/clean.
wildflower@wildflower:~/IA_Project$ |
```

2) Affichez les 5 premières lignes de train.csv

Commande(s) utilisée(s) : `head -5`

```
wildflower@wildflower:~/IA_Project$ head -5 datasets/brut/train.csv
id,age,revenu,ville,achat
1,25,350000,Dakar,Oui
2,42,720000,Thies,Non
3,31,500000,Saint-Louis,Oui
4,28,410000,Dakar,Oui
wildflower@wildflower:~/IA_Project$ |
```

3) Affichez les dernières lignes de application.log

Commande(s) utilisée(s) : `tail -3`

```
wildflower@wildflower:~/IA_Project$ tail -3 logs/application.log
2026-07-18 08:00:17 WARNING Valeurs manquantes détectées
2026-07-18 08:00:20 INFO Nettoyage terminé
2026-07-18 08:00:30 INFO Modèle prêt
wildflower@wildflower:~/IA_Project$ |
```

4) Comptez le nombre de lignes de clients.csv

Commande(s) utilisée(s) : `wc -l`

```
wildflower@wildflower:~/IA_Project$ wc -l datasets/brut/clients.csv
6 datasets/brut/clients.csv
wildflower@wildflower:~/IA_Project$ |
```

5) Recherchez toutes les lignes contenant WARNING

Commande(s) utilisée(s) : `grep -rn "WARNING"`

```
wildflower@wildflower:~/IA_Project$ grep -rn "WARNING" .
./logs/application.log:3:2026-07-18 08:00:17 WARNING Valeurs man
quantes détectées
wildflower@wildflower:~/IA_Project$
```

6) Recherchez le mot INFO dans les journaux

Commande(s) utilisée(s) : `grep -r "INFO" logs/`

```
wildflower@wildflower:~/IA_Project$ grep -r "INFO" logs/
logs/application.log:2026-07-18 08:00:10 INFO Application démarr
ée
logs/application.log:2026-07-18 08:00:15 INFO Chargement du data
set
logs/application.log:2026-07-18 08:00:20 INFO Nettoyage terminé
logs/application.log:2026-07-18 08:00:30 INFO Modèle prêt
wildflower@wildflower:~/IA_Project$ grep -rni "INFO" logs/
logs/application.log:1:2026-07-18 08:00:10 INFO Application déma
rrée
logs/application.log:2:2026-07-18 08:00:15 INFO Chargement du da
taset
logs/application.log:4:2026-07-18 08:00:20 INFO Nettoyage termin
é
logs/application.log:5:2026-07-18 08:00:30 INFO Modèle prêt
wildflower@wildflower:~/IA_Project$ |
```

7) Copiez train.csv dans datasets/clean

Commande(s) utilisée(s) : cp

```
wildflower@wildflower:~/IA_Project$ mkdir -p datasets/clean
wildflower@wildflower:~/IA_Project$ cp datasets/brut/train.csv datasets/clean/
wildflower@wildflower:~/IA_Project$ ls datasets/clean
train.csv
wildflower@wildflower:~/IA_Project$ |
```

8) Renommez model_info.txt en modele.txt

Commande(s) utilisée(s) : mv

```
wildflower@wildflower:~/IA_Project$ mv models/model_info.txt models/modele.txt
wildflower@wildflower:~/IA_Project$ ls models/
modele.txt
wildflower@wildflower:~/IA_Project$ |
```

9) Supprimez test.csv

Commande(s) utilisée(s) : rm

```
wildflower@wildflower:~/IA_Project$ rm datasets/brut/test.csv
wildflower@wildflower:~/IA_Project$ ls datasets/brut
clients.csv  readme.txt  train.csv
wildflower@wildflower:~/IA_Project$ |
```

10) Recherchez tous les fichiers .csv

Commande(s) utilisée(s) : find . -name "*.csv"

```
wildflower@wildflower:~/IA_Project$ find . -name "*.csv"
./datasets/brut/train.csv
./datasets/brut/clients.csv
./datasets/clean/train.csv
wildflower@wildflower:~/IA_Project$ |
```

11) Recherchez tous les fichiers .log

Commande(s) utilisée(s) : find . -name "*.log"

```
wildflower@wildflower:~/IA_Project$ find . -name "*.log"
./logs/training.log
./logs/application.log
wildflower@wildflower:~/IA_Project$ |
```

12) Affichez les fichiers triés par taille

Commande(s) utilisée(s) : `find . -type f -exec du -h {} \; | sort -h`

```
wildflower@wildflower:~/IA_Project$ find . -type f -not -path ".  
/.git/*" -exec du -h {} \; | sort -h  
4.0K      ./README.md  
4.0K      ./config/model.conf  
4.0K      ./config/settings.conf  
4.0K      ./datasets/brut/clients.csv  
4.0K      ./datasets/brut/readme.txt  
4.0K      ./datasets/brut/train.csv  
4.0K      ./datasets/clean/train.csv  
4.0K      ./logs/application.log  
4.0K      ./logs/global.log  
4.0K      ./logs/loss.log  
4.0K      ./logs/training.log  
4.0K      ./models/modele.txt  
4.0K      ./scripts/preprocess.py  
4.0K      ./scripts/train.py  
476K      ./rapport_atelier.pdf  
808K      ./backup/projet_IA.tar.gz  
wildflower@wildflower:~/IA_Project$ |
```

13) Concaténer application.log et training.log dans un nouveau fichier

Commande(s) utilisée(s) : `cat deux_fichier > fichier_recep`

```
wildflower@wildflower:~/IA_Project$ cat logs/application.log log  
s/training.log > logs/global.log  
wildflower@wildflower:~/IA_Project$ cat logs/global.log  
2026-07-18 08:00:10 INFO Application démarrée  
2026-07-18 08:00:15 INFO Chargement du dataset  
2026-07-18 08:00:17 WARNING Valeurs manquantes détectées  
2026-07-18 08:00:20 INFO Nettoyage terminé  
2026-07-18 08:00:30 INFO Modèle prêt  
Epoch 1 : Loss = 0.82  
Epoch 2 : Loss = 0.60  
Epoch 3 : Loss = 0.45  
Epoch 4 : Loss = 0.32  
Epoch 5 : Loss = 0.28  
Training terminé.  
wildflower@wildflower:~/IA_Project$ |
```


14) Extraire les lignes contenant Loss dans un fichier loss.log

Commande(s)

utilisée(s)

:

grep

-ri

```
wildflower@wildflower: ~/IA_ x + v - □ ×
./logs/global.log:Epoch 3 : Loss = 0.45
./logs/global.log:Epoch 4 : Loss = 0.32
./logs/global.log:Epoch 5 : Loss = 0.28
wildflower@wildflower:~/IA_Project$ grep -ri "loss" . > loss.log
grep: ./loss.log: input file is also the output
wildflower@wildflower:~/IA_Project$ grep -r "Loss" . --exclude-dir=.git
./logs/training.log:Epoch 1 : Loss = 0.82
./logs/training.log:Epoch 2 : Loss = 0.60
./logs/training.log:Epoch 3 : Loss = 0.45
./logs/training.log:Epoch 4 : Loss = 0.32
./logs/training.log:Epoch 5 : Loss = 0.28
./logs/global.log:Epoch 1 : Loss = 0.82
./logs/global.log:Epoch 2 : Loss = 0.60
./logs/global.log:Epoch 3 : Loss = 0.45
./logs/global.log:Epoch 4 : Loss = 0.32
./logs/global.log:Epoch 5 : Loss = 0.28
./loss.log:./logs/training.log:Epoch 1 : Loss = 0.82
./loss.log:./logs/training.log:Epoch 2 : Loss = 0.60
./loss.log:./logs/training.log:Epoch 3 : Loss = 0.45
./loss.log:./logs/training.log:Epoch 4 : Loss = 0.32
./loss.log:./logs/training.log:Epoch 5 : Loss = 0.28
./loss.log:./logs/global.log:Epoch 1 : Loss = 0.82
./loss.log:./logs/global.log:Epoch 2 : Loss = 0.60
./loss.log:./logs/global.log:Epoch 3 : Loss = 0.45
./loss.log:./logs/global.log:Epoch 4 : Loss = 0.32
./loss.log:./logs/global.log:Epoch 5 : Loss = 0.28
wildflower@wildflower:~/IA_Project$ ls
README.md datasets loss.log rapport_atelier.pdf
config logs models scripts
wildflower@wildflower:~/IA_Project$ mv loss.log logs/
wildflower@wildflower:~/IA_Project$ ls
README.md datasets models scripts
config logs rapport_atelier.pdf
wildflower@wildflower:~/IA_Project$ ls logs/
application.log global.log loss.log training.log
wildflower@wildflower:~/IA_Project$ |
```

15) Comptez le nombre de fichiers du projet

Commande(s) utilisée(s) : `find . -type f | wc -l` ou bien

```
find: unknown predicate '-'
wildflower@wildflower:~/IA_Project$ find . -type f | wc -l
70
wildflower@wildflower:~/IA_Project$ |
```

`find . -type f -not -path "/.git/*" | wc -l`

```
wildflower@wildflower:~/IA_Project$ find . -type f | wc -l
70
wildflower@wildflower:~/IA_Project$ find . -type f --exclude-dir
=.git | wc -l
find: unknown predicate '--exclude-dir=.git'
0
wildflower@wildflower:~/IA_Project$ find . -type f . --exclude-d
ir=.git | wc -l
find: paths must precede expression: `.'
find: possible unquoted pattern after predicate '-type'?
0
wildflower@wildflower:~/IA_Project$ find . -type f -not -path ".
/.git/*" | wc -l
16
```

16) Dans le dossier backup, créez projet_IA.tar.gz : la sauvegarde du projet

Commande(s) utilisée(s) : `tar czf chemin emplacement future du zip dossier a zip`

```
wildflower@wildflower:~/IA_Project$ ls
README.md  config  logs  rapport_atelier.pdf  wc
backup     datasets  models  scripts
wildflower@wildflower:~/IA_Project$ ls backup
wildflower@wildflower:~/IA_Project$ cd ~
wildflower@wildflower:~$ ls
IA_Project
wildflower@wildflower:~$ tar -czf IA_Project/backup/projet_IA.ta
r.gz IA_Project
tar: IA_Project/backup: file changed as we read it
wildflower@wildflower:~$ cd IA_Project
wildflower@wildflower:~/IA_Project$ ls
README.md  config  logs  rapport_atelier.pdf  wc
backup     datasets  models  scripts
wildflower@wildflower:~/IA_Project$ ls backup/
projet_IA.tar.gz
wildflower@wildflower:~/IA_Project$ |
```

17) Restaurez la sauvegarde ailleurs, en dehors du projet

Commande(s) utilisée(s) : `tar xzf chemin_fichier_a_dezip -C emplacement_du_dezip`

```
wildflower@wildflower:~$ ls restauration_test/
wildflower@wildflower:~$ tar -xzf ~/IA_Project/backup/projet_IA.
tar.gz -C ~/restauration_test
wildflower@wildflower:~$ ls restauration_test/
IA_Project
wildflower@wildflower:~$ ls restauration_test/IA_Project
README.md  config  logs  rapport_atelier.pdf  wc
backup     datasets  models  scripts
```

Partie 3 — Utilisateurs et permissions

1) Création des utilisateurs

Utilisateur	Fonction
alice	Data Scientist
bob	Data Engineer
charles	Développeur API
diane	MLOps Engineer
eva	Stagiaire

Pour chacun : créer un mot de passe, vérifier que son répertoire personnel a bien été créé, afficher son identifiant (UID) et ses groupes.

3.1) Créer les 5 utilisateurs et vérifier UID / dossier personnel / groupes

Commande(s) utilisée(s) : _____

 Insérer la capture d'écran ici

2) Création des groupes

Groupe	Rôle
data	Jeux de données
models	Modèles IA
api	Développement API
mlops	Déploiement
interns	Stagiaires

3.2) Créer les 5 groupes

Commande(s) utilisée(s) : _____

 Insérer la capture d'écran ici

3.3) Ajouter les utilisateurs dans leurs groupes respectifs

Commande(s) utilisée(s) : _____

 Insérer la capture d'écran ici

3.4) Créer les sous-dossiers api, backup, documentation, shared à la racine du projet

Commande(s) utilisée(s) : _____

 Insérer la capture d'écran ici

5) Propriétaires et groupes des dossiers

Dossier	Propriétaire	Groupe
datasets	alice	data
models	alice	data
api	charles	api
logs	diane	mlops
backup	diane	mlops
documentation	bob	data
shared	root	data

3.5) Attribuer les propriétaires et les groupes à chaque dossier

Commande(s) utilisée(s) : _____

 Insérer la capture d'écran ici

7) Droits par dossier

Dossier	Propriétaire	Groupe	Autres
datasets	rwX	rwX	---
models	rwX	r-X	---
api	rwX	rwX	---
logs	rwX	r--	---
backup	rwX	---	---
documentation	rwX	r--	r--
shared	rwX	rwX	r-X

3.6) Configurer les droits (chmod) selon le tableau ci-dessus

Commande(s) utilisée(s) : _____

 Insérer la capture d'écran ici

3.7) Vérifier les droits depuis le compte de chaque utilisateur (alice, bob, eva...)

Commande(s) utilisée(s) : _____

 Insérer la capture d'écran ici

3.8) Intégrer un nouveau collaborateur Frank (Data Scientist) : compte, groupes, accès datasets/shared, refus sur backup

Commande(s) utilisée(s) : _____

 Insérer la capture d'écran ici

Partie 4 — Installation des logiciels

4.1) Installez git, curl, wget, htop, tree, python3, python3-pip et unzip

Commande(s) utilisée(s) : _____

 Insérer la capture d'écran ici

4.2) Vérifiez leur installation

Commande(s) utilisée(s) : _____

 Insérer la capture d'écran ici

4.3) Téléchargez le dataset (iris.csv)

Commande(s) utilisée(s) : _____

 Insérer la capture d'écran ici

4.4) Vérifiez sa présence

Commande(s) utilisée(s) : _____

 Insérer la capture d'écran ici

4.5) Consultez son contenu

Commande(s) utilisée(s) : _____

 Insérer la capture d'écran ici

Partie 5 — Gestion des processus

5.1) Créez le script print-pause.py dans le sous-dossier scripts

Commande(s) utilisée(s) : _____

 Insérer la capture d'écran ici

5.2) Lancez le script

Commande(s) utilisée(s) : _____

 Insérer la capture d'écran ici

5.3) Observez son PID

Commande(s) utilisée(s) : _____

 Insérer la capture d'écran ici

5.4) Consultez son utilisation CPU

Commande(s) utilisée(s) : _____

 Insérer la capture d'écran ici

5.5) Arrêtez le processus

Commande(s) utilisée(s) : _____

 Insérer la capture d'écran ici

5.6) Relancez en arrière-plan

Commande(s) utilisée(s) : _____

 Insérer la capture d'écran ici

5.7) Remettez au premier plan

Commande(s) utilisée(s) : _____

 Insérer la capture d'écran ici

Partie 6 — Bash scripting

Le script `setup_project.sh` automatise l'ensemble de la préparation du projet : il demande le nom du projet, crée l'arborescence, génère le fichier de configuration, installe les outils, télécharge le dataset, compresse le projet et affiche un résumé final.

 *Le script complet, commenté, est fourni en fichier séparé (`setup_project.sh`) et constitue le premier livrable de l'atelier.*

6.1) Exécuter `setup_project.sh` et illustrer le résultat (résumé final affiché)

Commande(s) utilisée(s) : _____



Insérer la capture d'écran ici

Partie 7 — Bonus

Activité complémentaire proposée par le groupe :

7.1) Réaliser et illustrer l'activité bonus proposée

Commande(s) utilisée(s) : _____



Insérer la capture d'écran ici

Conclusion — Commandes principales utilisées

Court résumé des principales commandes Linux utilisées dans cet atelier et de leur rôle dans la préparation d'un environnement de développement IA :

Commande	Rôle dans le projet
mkdir -p	Créer l'arborescence du projet
cat / head / tail	Lire le contenu des fichiers de données et de logs
grep	Rechercher des informations (WARNING, INFO, Loss...)
cp / mv / rm	Organiser et nettoyer les fichiers du projet
adduser / usermod	Créer et gérer les comptes des membres de l'équipe
groupadd / gpasswd	Organiser les droits par rôle (data, api, mlops...)
chmod / chown	Sécuriser l'accès aux dossiers sensibles (backup...)
apt install	Installer les outils nécessaires au projet
curl / wget	Récupérer le dataset depuis Internet
ps / top / kill	Suivre et contrôler les traitements en cours
tar -czf	Sauvegarder l'ensemble du projet
bash script (.sh)	Automatiser toute la mise en place du projet